EECS 482
Introduction to operating systems

Semaphores and monitors

Peter Chen
University of Michigan

Producer-consumer with semaphores

- As always, think about shared data, mutual exclusion, and before-after constraints
- Semaphores:
  - *mtx*: for exclusive access to coke machine
  - *fullSlots*: counts number of cokes in machine
  - *emptySlots*: counts number of empty slots in machine
- Before-after constraints
  - Consumer must wait if no cokes in machine
    - » fullSlots.down()

    What if there's 1 full slot, and multiple consumers call down() at the same time?
  - Producer must wait if machine is full
    - » emptySlots.down()

Producer-consumer with semaphores

```
semaphore mtx(1)                // mutual exclusion to shared data
semaphore fullSlots(0)          // number of full slots
semaphore emptySlots(MAX)       // number of empty slots
```

```
producer {
  mtx.down()

  emptySlots.down()

  // add coke to machine

  fullSlots.up()

  mtx.up()

}
```

```
consumer {
  mtx.down()

  fullSlots.down()

  // take coke out of machine

  emptySlots.up()

  mtx.up()

}
```

What's wrong with this solution?

Producer-consumer with semaphores

```
semaphore mtx(1)                // mutual exclusion to shared data
semaphore fullSlots(0)          // number of full slots
semaphore emptySlots(MAX)       // number of empty slots
```

```
producer {
  mtx.down()

  emptySlots.down()

  // add coke to machine

  fullSlots.up()

  mtx.up()

}
```

```
consumer {
  mtx.down()

  fullSlots.down()

  // take coke out of machine

  emptySlots.up()

  mtx.up()

}
```

Any concerns?

Producer-consumer with semaphores

```
semaphore mtx(1)                // mutual exclusion to shared data
semaphore fullSlots(0)          // number of full slots
semaphore emptySlots(MAX)       // number of empty slots
```

```
producer {
  emptySlots.down()

  mtx.down()

  // add coke to machine

  mtx.up()

  fullSlots.up()

}
```

```
consumer {
  fullSlots.down()

  mtx.down()

  // take coke out of machine

  mtx.up()

  emptySlots.up()

}
```

Where should producer call fullSlots.up()?

Monitors vs. semaphores

- Monitors
  - Locks for mutual exclusion
  - Condition variables for ordering

- Semaphores: one mechanism for both mutual exclusion and ordering
  - Elegantly minimal
  - Can be difficult to use

Mutual exclusion (locks vs. semaphores)

- lock = binary semaphore (initialized to 1)
- lock() = down()
- unlock() = up()

Mutex	**Semaphore**

`mutex m`
 	
`m.lock()`
`<critical section>`
`m.unlock()`
	`semaphore m(1)`
	
	`m.down()`
	`<critical section>`
	`m.up()`

Ordering (condition variables vs. semaphores)

Condition variable	**Semaphore**
`while (condition) {wait(m)}`	`down()`
Waiting condition is in user code and uses user variables	Waiting condition is in semaphore code and uses semaphore's value
Waiting condition defined by user (more flexible)	Waiting condition defined by semaphore (wait if semaphore value==0)
User variables are protected by mutex	Semaphore's value is protected by semaphore's implementation of up(), down()
Must hold lock when calling wait	Must not hold "lock" (semaphore) when calling down() for ordering

Ordering (condition variables vs. semaphores)

Condition variable	**Semaphore**
No memory of past signals: Thread A: wait() Thread B: signal() What happens if wait(), then signal()? What happens if signal(), then wait()? Why is this ok?	Remembers past up calls Thread A: down() Thread B: up() What happens if down(), then up()? What happens if up(), then down()?

Implementing custom waiting condition with semaphores

- Semaphores work best if the shared integer and waiting condition (value==0) map naturally to problem domain

- How to implement custom waiting condition with semaphores?

What's a condition variable?

- A place for threads to wait
- "A place for threads"
  - A set of waiting threads
- "to wait"
  - To wait, create a semaphore(0), add the semaphore to the waiting set, then call down() on it
  - To signal, call up() on the waiting thread's semaphore

```
mutex cokeLock
cv waitingConsumers
cv waitingProducers
```

Consumer

```
cokeLock.lock()

while (numCokes == 0) {
   waitingConsumers.wait(cokeLock)
}

numCokes--

waitingProducers.signal()

cokeLock.unlock()
```

Producer

```
cokeLock.lock()

while (numCokes == MAX) {
    waitingProducers.wait(cokeLock)
}

numCokes++

waitingConsumers.signal()

cokeLock.unlock()
```

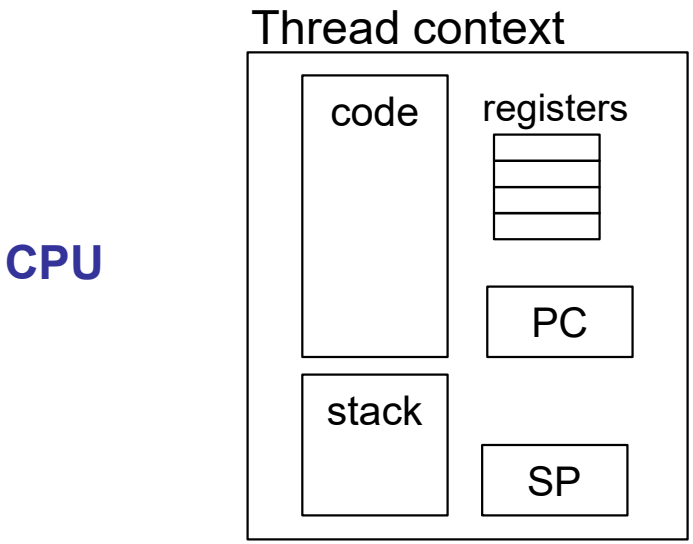
Threads can interact in two ways

- Multiple threads share data to cooperate on task
  - Coordinate via synchronization operations, e.g., locks, condition variables, semaphores
- We've been assuming that each thread has its own processor (of unpredictable speed). But actually…
- Multiple threads can share a single processor
  - Play analogy: one actor plays all the parts

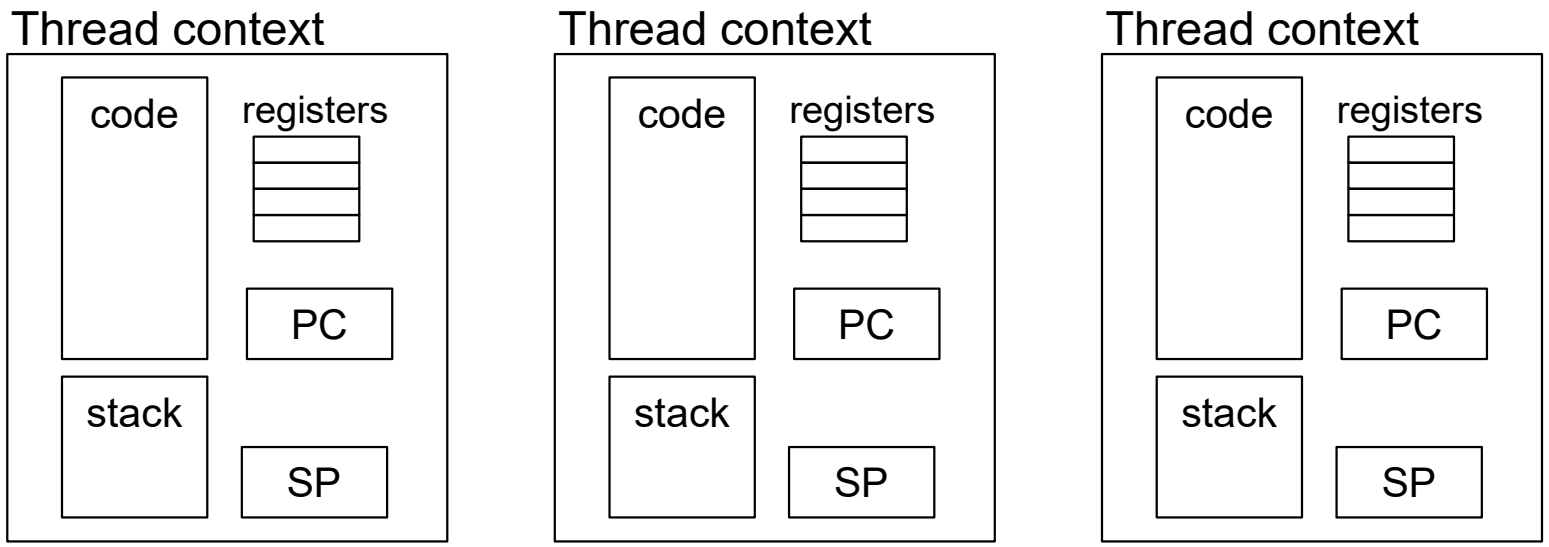
What's a thread?

- A thread is a sequence of executing instructions
- So what's a non-running thread?
  - A non-running thread is a paused execution
  - Can pausing a thread break a correct concurrent program?
- How to pause a thread and resume it later?

Per-thread state



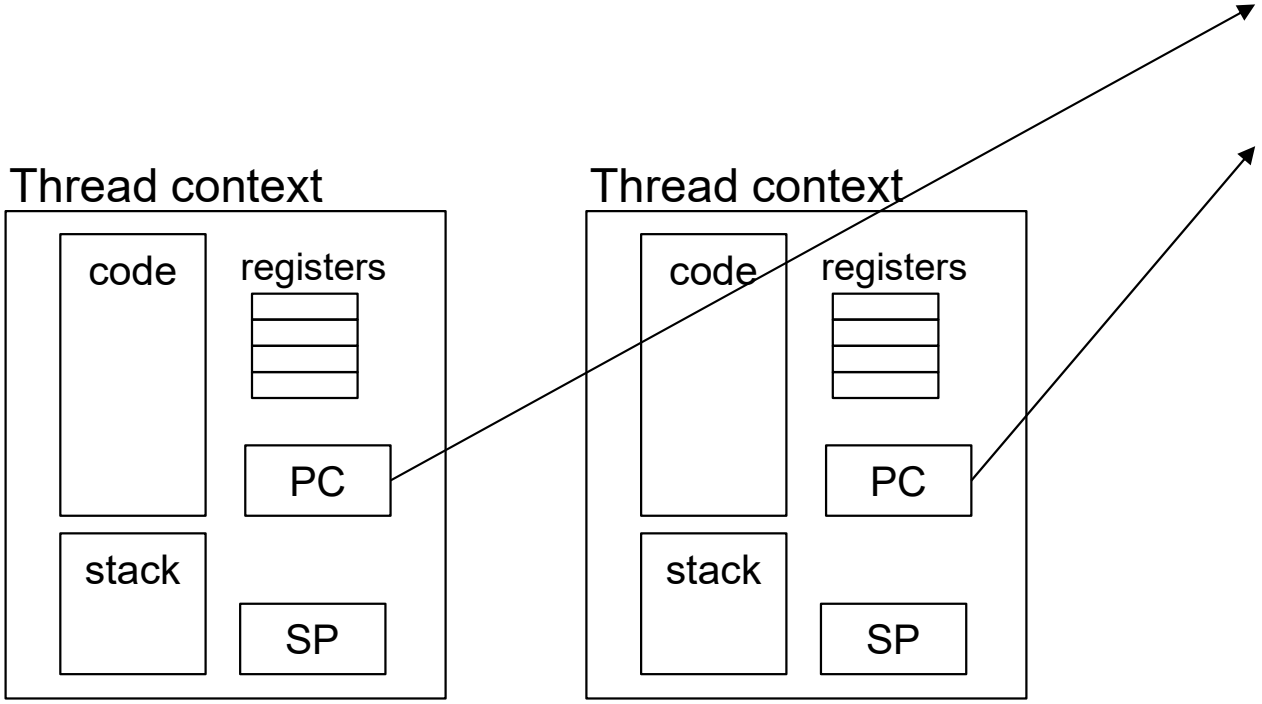
Sharing the CPU



CPU

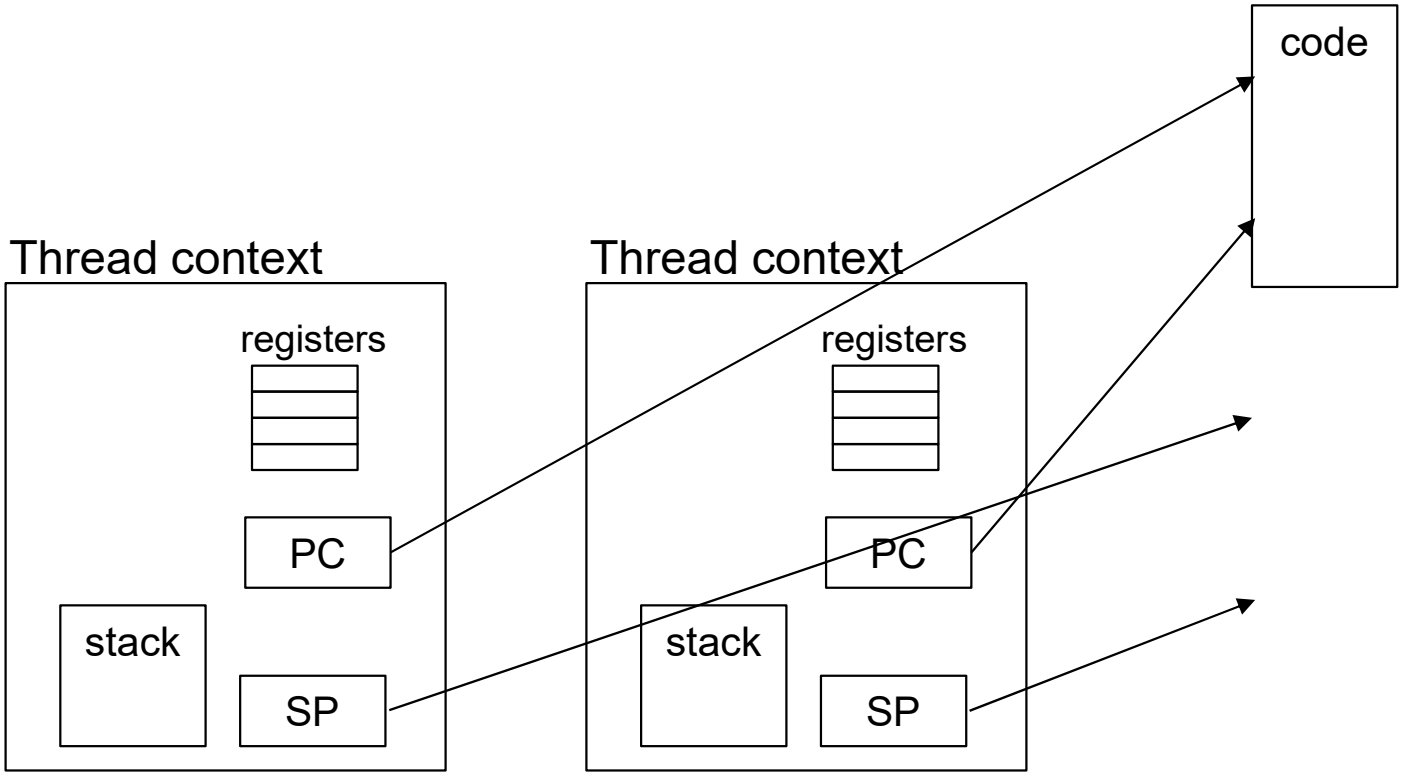
Optimizations

- How to avoid copying code back and forth?
  - Store separately from thread context (PC is still part of context)
  - Threads use same code

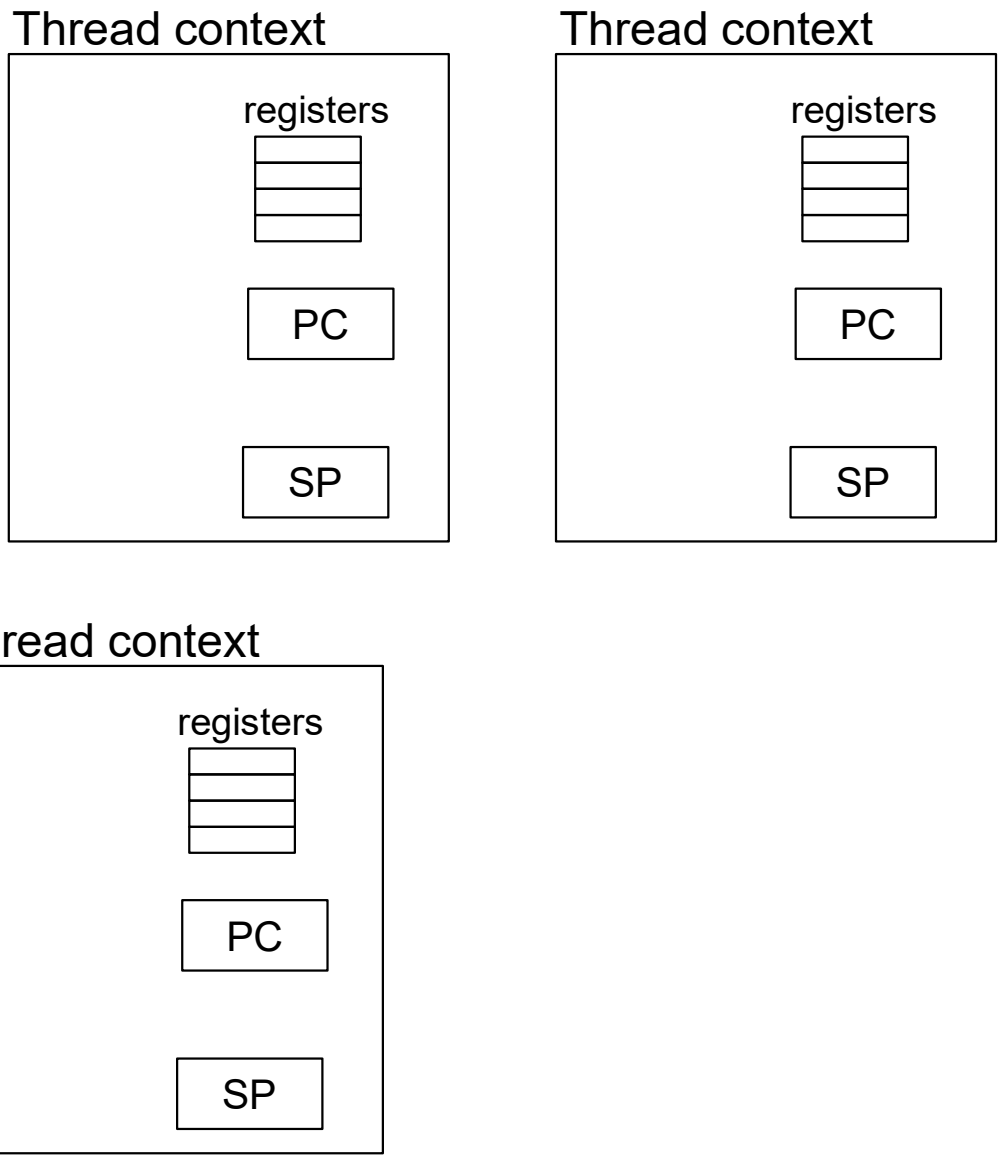


Optimizations

- How to avoid copying the stack?
  - Store separately from thread context (SP is still part of context)



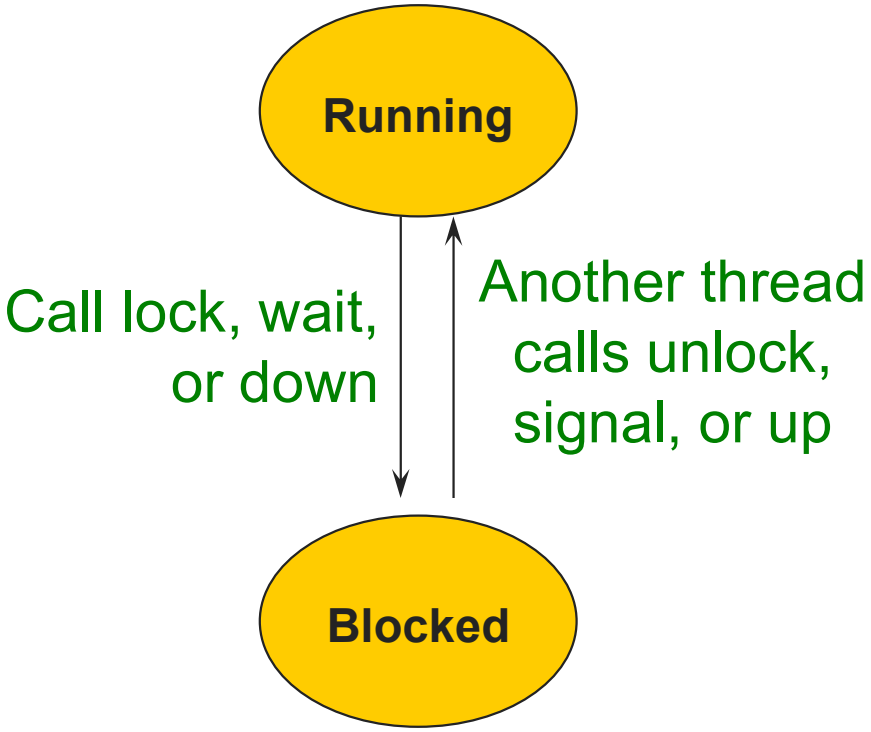
Sharing the CPU



Two perspectives when sharing a processor

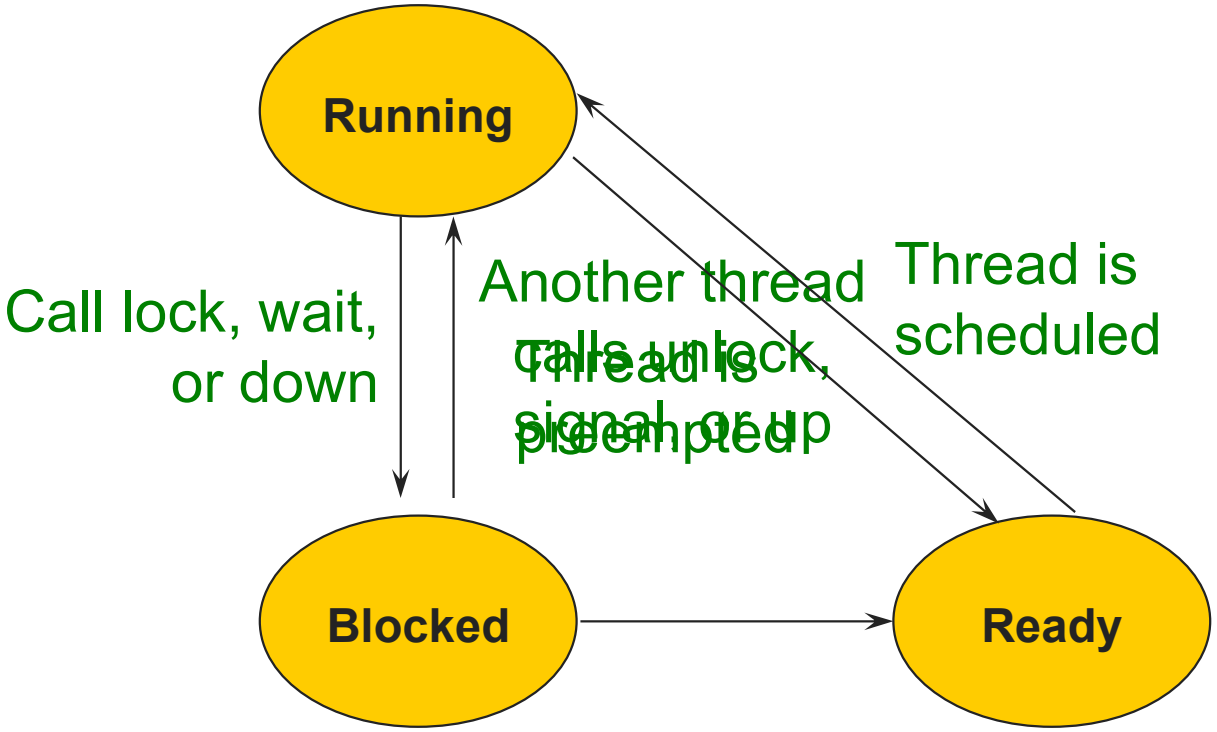
- Thread view:
  - Running → Paused → Running
- CPU view:
  - Thread 1 → Thread 2 → Thread 1
- First, let's take the thread view

States of a thread



What if there are more threads than CPUs?

States of a thread



Why no transition from Blocked to Running?

Why no transition from Ready to Blocked?